Stage 2 实验报告

于越洋

1 实验内容

Step 5

首先，我完成 `frontend/typecheck/namer.py` 中的 `Namer` 类的 `visitDeclaration`，`visitAssignment` 和 `visitIdentifier` 函数。

- `visitDeclaration`： 用于处理声明语句，首先检查要声明的变量是否已经被声明过，若未被声明过则声明新的变量，否则报告重声明错误。
- `visitAssignment`： 用于处理赋值语句，二元运算符处理函数 `visitBinary`，接收表达式的左右两个操作数。
- `visitIdentifier`： 用于处理标识符访问，若标识符已被声明过则正常访问，否则报告未声明错误。

接下来，我完成 `frontend/tacgen/tacgen.py` 中的 `TACGen` 类的三个与上面同名的函数。

- `visitDeclaration`： 为声明的新变量分配临时变量，若声明语句含有初始化值则使用赋值方法 `emitAssignment` 来处理。
- `visitAssignment`： 获取赋值语句对应的变量和值，由变量标识符找到其对应的临时变量，再用赋值方法 `emitAssignment` 来处理。
- `visitIdentifier`： 为标识符对应的变量标记所分配的临时变量。

最后，我在 `backend/riscv/riscvasmemitter.py` 中的 `RiscvInstrSelector` 类中添加 `visitAssign` 函数，将赋值语句编译为 `mv` 指令，以完成 TAC 到 RISC-V 汇编指令的转换。

Step 6

首先，我在 `frontend/scope` 目录下新建 `scopestack.py` 文件，其中包含一个新的类 `ScopeStack`，用于表示作用域栈。作用域栈包含多个“先进后出”的作用域 `Scope` 对象，并拥有 `push`， `pop`， `top`， `declare`， `lookup` 方法，以便使用。
然后，我将 `frontend/typecheck/namer.py` 和 `frontend/typecheck/typer.py` 中的访问者类泛型从 `Scope` 改为 `ScopeStack`，即将原来的单作用域改为作用域栈。随之我也修改了 `frontend/typecheck/namer.py` 中与作用域相关的 `transform`， `visitBlock`，`visitDeclaration`， `visitFunction` 函数。现在每当出现一个新的代码块和函数时，将会新建局部作用域并压栈，并在处理完成后弹栈；对于标识符的访问，在栈中所有作用域内寻找同名标识符；对于声明语句，在栈顶作用域中寻找同名标识符。由此实现了对多作用域和块语句的支持。

接下来，我在 `backend/dataflow/cfg.py` 的 `CFG` 类中添加了 `getReachable` 函数，用搜索算法求所有可达的作用域节点，并在初始化时调用。我还添加了 `reachable` 函数用于判断一个作用域是否可达，并在 `backend/reg/bruteregalloc/py` 内的 `BruteRegAlloc::accept` 函数中调用，用于跳过不可达的作用域块，不为其分配寄存器，从而提高寄存器的利用率。

2 思考题

Step 5

Question 1

> 我们假定当前栈帧的栈顶地址存储在 sp 寄存器中，请写出一段 **risc-v 汇编代码**，将栈帧空间扩大 16 字节。（提示1：栈帧由高地址向低地址延伸；提示2：risc-v 汇编中 addi reg0, reg1, <立即数> 表示将 reg1 的值加上立即数存储到 reg0 中。）

Answer：RISC-V 汇编代码为

```
addi sp, sp, -16
```

Question 2

> 有些语言允许在同一个作用域中多次定义同名的变量，例如这是一段合法的 Rust 代码（你不需要精确了解它的含义，大致理解即可）：
>
> ```
> fn main() {
>  let a = 0;
>  let a = f(a);
>  let a = g(a);
> }
> ```
>
> 其中 `f(a)` 中的 `a` 是上一行的 `let a = 0;` 定义的，`g(a)` 中的 `a` 是上一行的 `let a = f(a);` 。
> 如果 MiniDecaf 也允许多次定义同名变量，并规定新的定义会覆盖之前的同名定义，请问在你的实现中，需要对定义变量和查找变量的逻辑做怎样的修改？（提示：如何区分一个作用域中**不同位置**的变量定义？）

Answer：若要多次定义同名变量，可以在 `VarSymbol` 类中记录每个变量声明的顺序信息 `order`，比如使用声明变量的行数作为 `order`，以保证其单调递增，能够用于区分变量声明的先后顺序，然后将原来的 `Scope` 中变量名到 `VarSymbol` 的映射改成变量名到 `List[VarSymbol]` 的映射，从而按顺序存储同名变量的多个定义。

- 定义新变量时，直接在当前作用域的符号表中变量名对应的列表末尾添加一个新的 `VarSymbol` 对象；

- 访问变量时，直接返回列表中的最后一个 `VarSymbol` 对象；
- 如遇 `let a = f(a)` 这样的声明语句，先处理等号右侧的表达式，记录右侧访问的变量 `a` 对应的 `order` 值，再处理等号左侧的变量标识符，声明新的变量 `a`，这样，在生成 TAC 时，对于等号右侧的 `a`，只需在列表中寻找与先前记录 `order` 值一致的的 `VarSymbol` 对象即可。

Step 6

> 请画出下面 MiniDecaf 代码的控制流图。

```
int main() {
    int a = 2;
    if (a < 3) {
        {
            int a = 3;
            return a;
        }
        return a;
    }
}
```

Answer：若直接探讨以上 Minidecaf 源代码的控制流，记控制流块 `B1` 为声明语句 `int a = 2;`，`B2` 为判断语句 `if (a < 3)`，`B3` 为代码块 `{ int a = 3; return a; }`，`B4` 为返回语句 `return a;`，`B5` 为程序正常结束（即 `return 0;`），`Exit` 表示程序结束运行，则控制流图为

mermaid-diagram-2025-10-20-002848.png > pic_center

若探讨中间代码的控制流，首先生成中间代码为

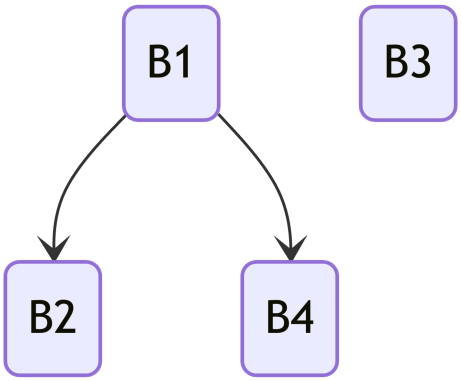
```
FUNCTION<main>:
    _T1 = 2
    _T0 = _T1
    _T2 = 3
    _T3 = (_T0 < _T2)
    if (_T3 == 0) branch _L1
    _T5 = 3
    _T4 = _T5
    return _T4
    return _T0
_L1:
    return
```

可以分成以下 4 个基本块：

```
------ B1 ------
FUNCTION<main>:
    _T1 = 2
    _T0 = _T1
    _T2 = 3
    _T3 = (_T0 < _T2)
    if (_T3 == 0) branch _L1

------ B2 ------
_T5 = 3
    _T4 = _T5
    return _T4

------ B3 ------
    return _T0

------ B4 ------
_L1:
    return
```

控制流图为